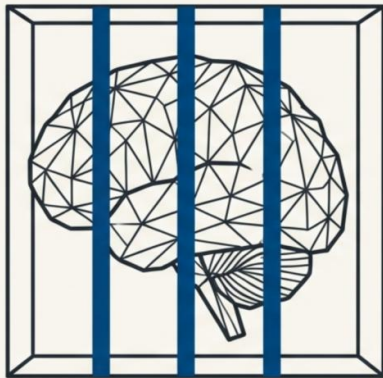


생성형 AI 기반 주식 투자 에이전트 구축

1. 서론: LLM 의 한계와 에이전트 시스템의 필요성

1.1 대규모 언어 모델(LLM)의 본질적 한계와 '감옥'의 비유

감혀있는 천재: LLM의 본질적 한계



GPT-4o와 같은 최신 LLM은 놀라운 추론 능력을 보여주지만, 근본적인 한계에 감혀 있습니다.

핵심 비유: "시계도 창문도 없는 감옥에 갇힌 천재"

이 비유는 LLM의 두 가지 결정적인 결핍을 설명합니다.



1. 시간적 단절 (Temporal Disconnection)

내장된 시계(System Clock)가 없어 현재 시간을 알지 못합니다. "지금 몇 시야?"라는 질문에 환각(Hallucination)을 일으킬 수 있습니다.



2. 외부 세계와의 단절 (External Disconnection)

인터넷, 데이터베이스 등 외부 정보에 직접 접근할 수 없습니다. "오늘 비가 오니?"와 같은 질문에 답하려면 외부 세계와의 연결이 필요합니다."

현대의 인공지능, 특히 GPT-4o 와 같은 대규모 언어 모델(LLM)은 텍스트 생성과 추론 능력에서 놀라운 성능을 보여주고 있습니다. 그러나 이들 모델은 근본적인 구조적 한계를 지니고 있습니다. 본 강의 문서의 기반이 되는 연구 자료에서는 GPT 모델을 "시계도 창문도 없는 감옥에 갇혀 있는 사람"에 비유합니다.¹ 이 비유는 LLM 의 두 가지 결정적인 결핍을 설명합니다.

첫째, ****시간적 단절(Temporal Disconnection)****입니다. LLM 은 학습이 종료된 시점(Cut-off date)까지의 데이터만을 기억합니다. 모델 그 자체는 현재가 몇 년 몇 월 며칠인지, 지금 시각이 몇 시인지 알 수 있는 내부 시계(System Clock)를 가지고 있지 않습니다. 따라서 사용자가 "지금 몇 시야?"라고 물었을 때, 순수한 LLM 은 "죄송하지만 저는 실시간 정보를 알 수 없습니다"라고 대답하거나, 학습 데이터에 존재하는 과거의 시간을 그럴듯하게 꾸며내는 환각(Hallucination) 현상을 일으킬 수 있습니다.¹

둘째, ****외부 세계와의 단절(External Disconnection)****입니다. LLM 은 인터넷에 직접 접속하거나, 외부 데이터베이스를 조회하거나, 복잡한 수학 연산을 수행하는 계산기에 접근할 수 있는 권한이

기본적으로 없습니다. "오늘 비가 오니?"라는 질문에 대답하기 위해서는 창문을 열어 밖을 보거나 기상청 서버에 접속해야 하지만, API 를 통해 제공되는 기본 모델은 이러한 '창문'이나 '네트워크'를 갖추지 못한 상태입니다.

1.2 에이전트(Agent)로의 진화와 평선 콜링(Function Calling)

Premium technology report

감옥을 탈출하는 열쇠, 평선 콜링

이 한계를 극복하는 핵심 기술이 바로 **AI 에이전트**와 **평선 콜링(Function Calling)**입니다.

평선 콜링은 LLM에게 외부 도구(함수)를 사용할 수 있는 능력을 부여합니다.
마치 감옥 안의 천재에게 외부와 소통할 수 있는 '전화기'를 주는 것과 같습니다.



작동 원리

1. **요청 (Request):** LLM이 사용자의 질문을 분석하여 필요한 도구가 있는지 판단하고, 함수 실행을 요청합니다.
2. **실행 (Execution):** 애플리케이션이 LLM의 요청을 받아 실제 파이썬 코드를 실행합니다.
3. **응답 (Response):** 코드 실행 결과를 다시 LLM에게 전달합니다.
4. **최종 답변 생성:** LLM은 실행 결과를 바탕으로 자연스러운 최종 답변을 만듭니다.

이러한 한계를 극복하기 위해 등장한 개념이 바로 'AI 에이전트'이며, 이를 기술적으로 구현하는 핵심 메커니즘이 오픈 AI(OpenAI)의 **평선 콜링(Function Calling)**입니다. 평선 콜링은 LLM 에게 외부 도구(함수)를 사용할 수 있는 능력을 부여하는 기술입니다. 이는 마치 감옥에 갇혀 있는 천재에게 외부와 소통할 수 있는 전화기를 설치해 주는 것과 같습니다.

평선 콜링의 작동 원리는 다음과 같습니다. 개발자는 미리 특정 기능을 수행하는 함수(예: `get_current_time()`)를 정의하고, 이 함수에 대한 설명서(이름, 기능, 필요한 매개변수 등)를 LLM 에게 제공합니다. 사용자가 질문을 던지면, LLM 은 자신의 지식만으로 답변할지, 아니면 제공된 도구 중 하나를 사용할지 판단합니다. 도구를 사용하기로 결정하면, LLM 은 직접 코드를 실행하는 대신 "이 함수를 이러한 매개변수로 실행해 달라"는 요청(Request)을 생성합니다. 애플리케이션은 이 요청을 받아 실제 코드를 실행하고, 그 결과를 다시 LLM 에게 알려줍니다. LLM 은 이 실행 결과를 바탕으로 최종적인 답변을 생성합니다.¹

이 과정을 통해 LLM 은 단순한 텍스트 생성기를 넘어, 실시간 데이터를 조회하고 주식 시장을 분석하며 복잡한 의사결정을 지원하는 지능형 투자 비서로 진화할 수 있습니다. 본 보고서에서는

이러한 이론적 배경을 바탕으로, 파이썬(Python)과 오픈 AI API, 그리고 금융 데이터 라이브러리인 yfinance 를 활용하여 실제로 작동하는 AI 주식 투자자를 구축하는 전체 과정을 심도 있게 다룹니다.

2. 평션 콜링의 기초와 시간 정보 획득 시스템 구현

2.1 기초 환경 설정과 단순 시간 확인 기능

AI 에이전트 개발의 첫 단계는 가장 기본적인 실시간 정보인 '현재 시간'을 확인하는 기능을 구현하는 것입니다. 이는 AI가 외부 세계의 상태(State)를 인식하는 가장 기초적인 단계입니다.

2.1.1 파이썬 함수 정의 (gpt_functions.py)

먼저, 우리는 AI가 사용할 실제 도구를 만들어야 합니다. 연구 자료에서는 gpt_functions.py 라는 별도의 파일을 생성하여 함수를 관리하는 것을 권장합니다. 이는 코드의 모듈화를 통해 유지보수성을 높이기 위함입니다.

첫걸음: AI에게 시간의 개념을 알려주기

1. 실제 도구 정의 (`gpt\_functions.py`)

```
from datetime import datetime

def get_current_time():
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    return now
```

LLM에게 불가능한 '현재성'을 제공하는 핵심 코드. 시스템의 현재 시각을 가져옵니다.

2. 도구 설명서 정의 (`tools`)

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_time",
            "description": "현재 날짜와 시간을 반환합니다."
        }
    }
]
```

가장 중요한 부분.
LLM은 이 설명을 읽고 "지금 몇 시야?" 같은 질문에 어떤 함수를 사용할지 결정합니다.

get_functions.py

```
from datetime import datetime

def get_current_time():
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(now)
    return now

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_time",
            "description": "현재 날짜와 시간을 반환합니다.",
        }
    },
]

if __name__ == '__main__':
    get_current_time()
```

이 함수는 매우 간단하지만, LLM 에게는 불가능한 '현재성'을 제공하는 핵심적인 역할을 합니다.

`datetime.now()`를 호출하여 시스템의 현재 시각을 가져오고, 이를 문자열 포맷으로 변환하여 반환합니다. 여기서 `print(now)`를 추가한 이유는 개발자가 터미널에서 함수가 실제로 호출되었는지 시각적으로 디버깅하기 위함입니다.¹

2.1.2 도구 설명서(Schema) 정의

함수를 정의했다고 해서 GPT 가 바로 이를 사용할 수 있는 것은 아닙니다. GPT 에게 이 함수의 존재와 사용법을 알려주기 위해, 오픈 AI 가 정한 규격에 맞춰 도구 목록(tools)을 정의해야 합니다. 이 목록은 딕셔너리 형태의 리스트로 구성됩니다.¹

```
tools = [
    {
```

```

    "type": "function",
    "function": {
        "name": "get_current_time",
        "description": "현재 날짜와 시간을 반환합니다.",
    },
},
]

```

여기서 각 필드의 의미는 매우 중요합니다:

- **type**: 도구의 유형을 정의합니다. 현재는 "function"이 표준입니다.
- **name**: GPT가 호출할 함수의 정확한 이름을 명시합니다. 이 이름은 파이썬 함수명과 일치해야 매핑이 용이합니다.
- **description**: **가장 중요한 부분**입니다. GPT는 이 설명을 읽고 사용자의 질문과 대조하여 이 함수를 사용할지 말지를 결정합니다. "현재 날짜와 시간을 반환합니다"라는 설명은 "지금 몇 시야?"라는 질문과 의미적으로 연결됩니다.
- **parameters**: 함수에 필요한 매개변수를 정의하는 부분입니다. `get_current_time`은 매개변수가 필요 없으므로 이 초기 버전에서는 생략되었습니다.¹

2.2 터미널 기반의 챗봇 구현과 메시지 루프

도구가 준비되었다면, 이를 활용하는 메인 애플리케이션(`02_what_time_is_it_terminal.py`)을 구축해야 합니다. 이 단계에서는 사용자와 AI 간의 대화 루프(Loop)를 생성하고, 평션 쿨링의 요청을 처리하는 로직을 구현합니다.

2.2.1 API 호출 구조의 변경

기존의 단순한 챗봇과 달리, 평션 쿨링을 지원하는 챗봇은 `tools` 매개변수를 API 호출 시 함께 전달해야 합니다.

```

from gpt_functions import get_current_time, tools

#... (중략)...

```

```

def get_ai_response(messages, tools=None):

```

```

response = client.chat.completions.create(
    model="gpt-4o",
    messages=messages,
    tools=tools, # 도구 목록 전달
)

return response

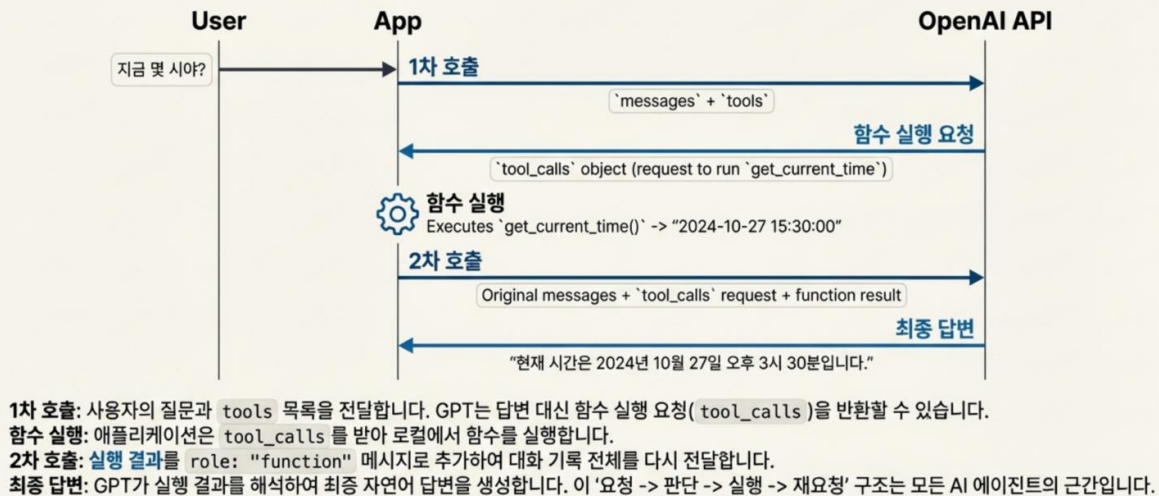
```

여기서 중요한 점은 반환값을 `response.choices.message.content` 와 같이 문자열만 추출하는 것이 아니라, `response` 객체 전체를 반환하도록 설계해야 한다는 것입니다. 왜냐하면, GPT 가 함수 실행을 요청할 경우 응답 내용은 비어있고(`None`), 대신 `tool_calls` 라는 속성에 실행 정보가 담겨 오기 때문입니다.¹

2.2.2 평선 콜링 처리 로직

보이지 않는 대화: 평선 콜링의 멀티턴 API 호출 구조

평선 콜링은 단 한 번의 API 호출로 끝나지 않습니다.



사용자의 입력부터 AI의 최종 답변까지의 흐름은 다음과 같은 정교한 제어문으로 구성됩니다.

1. **사용자 입력:** 사용자의 메시지를 `messages` 리스트에 추가합니다.
2. **1차 API 호출:** GPT에게 메시지와 도구 목록을 보냅니다.
3. **도구 호출 확인:** 응답 객체에서 `tool_calls` 속성이 존재하는지 확인합니다.
4. **함수 실행 및 결과 반환:**

- GPT 가 함수 실행을 요청했다면(tool_calls 존재), 해당 함수의 이름을 확인합니다.
- 이름이 get_current_time 이라면, 로컬에 정의된 파이썬 함수를 실행합니다.
- **중요:** 실행 결과는 반드시 messages 리스트에 새로운 메시지로 추가해야 합니다. 이때 메시지의 역할(role)은 "function"이어야 하며, tool_call_id 와 name 을 정확히 명시해야 GPT 가 어떤 요청에 대한 응답인지 식별할 수 있습니다.¹

5. **2 차 API 호출:** 함수 실행 결과가 포함된 대화 기록을 다시 GPT 에게 보냅니다. GPT 는 이 데이터를 바탕으로 "현재 시간은 2025 년 1 월 21 일 오후 4 시 35 분입니다"와 같은 자연어 답변을 생성합니다.

이러한 '요청 -> 판단 -> 실행 -> 재요청'의 멀티턴(Multi-turn) 구조는 모든 에이전트 시스템의 근간이 됩니다.

2.3 타임존(Timezone) 문제 해결과 매개변수화

더 똑똑한 에이전트: 매개변수로 타임존 문제 해결하기

- **Problem:** "뉴욕은 지금 몇 시야?"라는 질문에 서버 시간 기준인 한국 시간을 알려주면 안 됩니다.
- **Solution:** `timezone` 매개변수를 추가하여 함수를 개선합니다.

`get\_current\_time()` - BEFORE

```
def get_current_time():
    # ... returns server time
```

`get\_current\_time(timezone)` - AFTER

```
import pytz

def get_current_time(timezone='Asia/Seoul'):
    tz = pytz.timezone(timezone)
    now = datetime.now(tz)
    # ... returns timezone-aware time
```

Key Insight:

- 도구 설명서(tools)에도 'parameters' 필드를 추가해야 합니다.
- **LLM의 추론 능력:** 사용자가 "뉴욕 시간"이라고 말하면, LLM은 '뉴욕'이 'America/New_York'이라는 IANA 타임존 코드에 해당함을 **스스로 추론**하여 매개변수로 전달합니다. 개발자가 직접 매핑 테이블을 만들 필요가 없습니다.

초기 구현은 사용자의 위치를 고려하지 않고 서버(실행 환경)의 시간을 반환한다는 한계가 있습니다. 사용자가 "뉴욕은 지금 몇 시야?"라고 물었을 때, GPT 는 자신이 위치를 알 수 없으므로 엉뚱한 시간을 대답하거나, 그리니치 표준시(UTC)를 기준으로 잘못된 계산을 할 수 있습니다.¹

2.3.1 pytz 라이브러리를 활용한 개선

이를 해결하기 위해 pytz 라이브러리를 도입하여 타임존을 처리하도록 함수를 개선합니다.

gpt_functions.py

```
from datetime import datetime
import pytz

def get_current_time(timezone: str = 'Asia/Seoul'):
    tz = pytz.timezone(timezone) # 타임존 설정
    now = datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")
    now_timezone = f'{now} {timezone}'
    print(now_timezone)
    return now_timezone

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_time",
            "description": "해당 타임존의 날짜와 시간을 반환합니다.",
            "parameters": {
                "type": "object",
                "properties": {
                    'timezone': {
                        'type': 'string',
                        'description': '현재 날짜와 시간을 반환할 타임존을 입력하세요. (예: Asia/Seoul)',
                    }
                }
            }
        }
    },
]
```

```

        "required": ['timezone'],
    },
}
},
]

```

```
get_current_time('America/New_York')
```

이제 함수는 timezone 이라는 매개변수를 받아, 해당 지역의 시간을 정확히 계산할 수 있게 되었습니다.

2.3.2 스키마 업데이트와 GPT 의 추론 능력

함수가 변경되었으므로 도구 설명서(tools)도 업데이트해야 합니다. parameters 필드에 timezone 에 대한 정의를 추가합니다.

```

"parameters": {
  "type": "object",
  "properties": {
    "timezone": {
      "type": "string",
      "description": "현재 날짜와 시간을 반환할 타임존 (예: Asia/Seoul)",
    },
  },
  "required": ["timezone"],
}

```

이 부분이 시사하는 바는 매우 큼니다. 사용자가 "뉴욕 시간 알려줘"라고 자연어로 말하면, GPT 는 내부적인 지식을 동원하여 '뉴욕'이라는 지명이 America/New_York 이라는 IANA 타임존 코드에 해당함을 스스로 추론합니다. 그리고 함수 호출 시 이 코드를 인자(argument)로 전달합니다. 즉, **개발자가 별도로 지명-타임존 매핑 테이블을 만들 필요 없이, LLM 의 언어 능력을 활용하여 복잡한 전처리 과정을 생략할 수 있게 되는 것입니다.**¹

2.4 다중 함수 호출(Parallel Function Calling) 처리

한번에 여러 질문하기: 병렬 평선 콜링 처리

사용자가 “도쿄, 서울, 뉴욕의 현재 시간은?”과 같이 한 번에 여러 작업을 요청하면 어떻게 될까요?
LLM는 효율성을 위해 한 번의 응답에 여러 개의 함수 호출 요청(tool_calls)을 담아 보낼 수 있습니다.

✗ 잘못된 구현 (Incorrect)

```
# Processes only the first tool call
tool_call = response.choices[0].message
    .tool_calls[0]
# ...
```

첫 번째 'tool_call'만 처리하여 나머지 요청을 놓칩니다.

✓ 올바른 구현 (Correct)

```
# Loops through all tool calls
if tool_calls:
    for tool_call in tool_calls:
        tool_name = tool_call.function.name
        # ... (함수 실행 및 결과 추가 로직)
```

반복문을 사용하여 모든 'tool_calls'를 안정적으로 처리합니다.

Takeaway: 코드는 항상 여러 'tool_calls'가 반환될 가능성을 염두에 두고, **반복문(Loop)** 구조로 작성해야 합니다. 이를 통해 여러 도시의 시간을 한 번에 조회하고 **종합적인 답변을 생성**할 수 있습니다.

사용자가 "도쿄, 서울, 뉴욕의 시간을 알려줘"라고 한 번에 여러 지역을 물어볼 경우, GPT는 효율성을 위해 한 번의 응답에 여러 개의 함수 호출 요청을 포함시킬 수 있습니다. 초기 코드에서 tool_calls 과 같이 첫 번째 요청만 처리하도록 작성했다면, 나머지 도시의 시간은 무시되거나 오류가 발생할 것입니다.¹

따라서 코드는 반드시 반복문(Loop) 구조로 개선되어야 합니다:

```
if tool_calls:
```

```
    for tool_call in tool_calls:
```

```
        tool_name = tool_call.function.name
```

```
        #... (함수 실행 및 메시지 추가)
```

이 반복문을 통해 챗봇은 도쿄, 서울, 뉴욕에 대한 get_current_time 함수를 각각 실행하고, 세 가지 결과를 모두 메시지 기록에 추가한 뒤, 마지막으로 GPT에게 전달하여 종합적인 답변을 생성하게 합니다. 연구 자료에서는 이 과정에서 Traceback 오류나 문자열 연결 오류(TypeError)가 발생할 수 있음을 경고하며, 견고한 루프 작성의 중요성을 강조합니다.¹

what_time_is_it_terminal.py

```
from gpt_functions import get_current_time, tools
from openai import OpenAI
from dotenv import load_dotenv
import os
import json

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # 오픈 AI 클라이언트의 인스턴스 생성

def get_ai_response(messages, tools=None):
    response = client.chat.completions.create(
        model="gpt-4o", # 응답 생성에 사용할 모델 지정
        messages=messages, # 대화 기록을 입력으로 전달
        tools=tools, # 사용 가능한 도구 목록 전달
    )
    return response # 생성된 응답 내용 반환

messages = [
    {"role": "system", "content": "너는 사용자를 도와주는 상담사야."}, # 초기 시스템 메시지
]

while True:
    user_input = input("사용자>: ") # 사용자 입력 받기
    if user_input == "exit": # 사용자가 대화를 종료하려는지 확인
        break
```

```
messages.append({"role": "user", "content": user_input}) # 사용자 메시지 대화 기록에  
추가
```

```
ai_response = get_ai_response(messages, tools=tools)
```

```
ai_message = ai_response.choices[0].message
```

```
print(ai_message) # ③ gpt 에서 반환되는 값을 파악하기 위해 임시로 추가
```

```
tool_calls = ai_message.tool_calls # AI 응답에 포함된 tool_calls 를 가져옵니다.
```

```
if tool_calls: # tool_calls 가 있는 경우
```

```
    for tool_call in tool_calls:
```

```
        tool_name = tool_call.function.name # 실행해야한다고 판단한 함수명 받기
```

```
        tool_call_id = tool_call.id # tool_call 아이디 받기
```

```
        arguments = json.loads(tool_call.function.arguments) # (1) 문자열을 딕셔너리로
```

변환

```
        if tool_name == "get_current_time": # ⑤ 만약 tool_name 이  
"get_current_time"이라면
```

```
            messages.append({
```

```
                "role": "function", # role 을 "function"으로 설정
```

```
                "tool_call_id": tool_call_id,
```

```
                "name": tool_name,
```

```
                "content": get_current_time(timezone=arguments['timezone']), # 타임존 추가
```

```
            })
```

```
            messages.append({"role": "system", "content": "이제 주어진 결과를 바탕으로 답변할  
차례다."}) # 함수 실행 완료 메시지 추가
```

```
            ai_response = get_ai_response(messages, tools=tools) # 다시 GPT 응답 받기
```

```
            ai_message = ai_response.choices[0].message
```

```
messages.append(ai_message) # AI 응답을 대화 기록에 추가하기
```

```
print("AIWt: " + ai_message.content) # AI 응답 출력
```

what_time_is_it_streamlit.py

```
from gpt_functions import get_current_time, tools
from openai import OpenAI
from dotenv import load_dotenv
import os
import json
import streamlit as st

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # 오픈 AI 클라이언트의 인스턴스 생성

def get_ai_response(messages, tools=None):
    response = client.chat.completions.create(
        model="gpt-4o", # 응답 생성에 사용할 모델 지정
        messages=messages, # 대화 기록을 입력으로 전달
        tools=tools, # 사용 가능한 도구 목록 전달
    )
    return response # 생성된 응답 내용 반환

st.title("💬 Chatbot")

if "messages" not in st.session_state:
```

```

st.session_state["messages"] = [
    {"role": "system", "content": "너는 사용자를 도와주는 상담사야."}, # 초기 시스템 메시지
]

for msg in st.session_state.messages:
    if msg["role"] == "assistant" or msg["role"] == "user": # assistant 혹은 user 메시지인
        경우만
            st.chat_message(msg["role"]).write(msg["content"])

if user_input := st.chat_input(): # 사용자 입력 받기
    st.session_state.messages.append({"role": "user", "content": user_input}) # 사용자
    메시지를 대화 기록에 추가
    st.chat_message("user").write(user_input) # 사용자 메시지를 브라우저에서도 출력

    ai_response = get_ai_response(st.session_state.messages, tools=tools)
    ai_message = ai_response.choices[0].message
    print(ai_message) # gpt 에서 반환되는 값을 파악하기 위해 임시로 추가

    tool_calls = ai_message.tool_calls # AI 응답에 포함된 tool_calls 를 가져옵니다.
    if tool_calls: # tool_calls 가 있는 경우
        for tool_call in tool_calls:
            tool_name = tool_call.function.name # 실행해야한다고 판단한 함수명 받기
            tool_call_id = tool_call.id # tool_call 아이디 받기
            arguments = json.loads(tool_call.function.arguments) # 문자열을 딕셔너리로
            변환

            if tool_name == "get_current_time": # 만약 tool_name 이 "get_current_time"이라면
                st.session_state.messages.append({
                    "role": "function", # role 을 "function"으로 설정

```

```

        "tool_call_id": tool_call_id,
        "name": tool_name,
        "content": get_current_time(timezone=arguments['timezone']), # 타임존 추가
    })

    st.session_state.messages.append({"role": "system", "content": "이제 주어진 결과를 바탕으로 답변할 차례다."})

    ai_response = get_ai_response(st.session_state.messages, tools=tools) # 다시 GPT 응답 받기
    ai_message = ai_response.choices[0].message

    st.session_state.messages.append({
        "role": "assistant",
        "content": ai_message.content
    }) #AI 응답을 대화 기록에 추가합니다.

    print("AIwt: " + ai_message.content) # AI 응답 출력
    st.chat_message("assistant").write(ai_message.content) # 브라우저에 메시지 출력

```



Chatbot



안녕 누구야?



안녕하세요! 저는 여러분을 도와드리기 위한 AI 상담사입니다. 무엇을 도와드릴까요?



지금 몇시야?



어느 지역의 시간을 알고 싶으신가요? 타임존을 알려주시면 그곳의 현재 시간을 알려드릴 수 있습니다. 예: 서울의 경우 "Asia/Seoul"이라고 말씀해 주세요.



서울, 멜버른, 뉴욕



현재 시간은 다음과 같습니다:

- 서울: 2025년 12월 11일 17시 35분
- 멜버른: 2025년 12월 11일 19시 35분
- 뉴욕: 2025년 12월 11일 03시 35분

다른 도움이 필요하신가요?

Your message



3. 심화: 실전 금융 데이터 분석 에이전트 구축

3.1 yfinance 라이브러리와 데이터 엔지니어링

실전 투입: AI 주식 분석가 만들기

이제 에이전트에게 실제 가치를 창출하는 금융 분석 능력을 부여합니다.

핵심 도구: **yfinance** 라이브러리 (야후 파이낸스 데이터 제공)



기업 기본 정보

`get_yf_stock_info(ticker)`

“테슬라는 어떤 회사야?” -> 기업 개요,
시가총액, PER 등 상세 정보 제공.



주가 히스토리

`get_yf_stock_history(ticker,
period)`

“최근 한 달간 애플 주가 보여줘.” ->
기간별 OHLCV 주가 데이터 반환.



투자 의견

`get_yf_stock_recommendations(
ticker)`

“지금 엔비디아 사도 될까?” ->
전문가들의 ‘강력 매수’, ‘보유’ 등
투자 의견 분포 제공.

시간 확인 기능을 넘어, 실제 주식 투자 정보를 제공하기 위해서는 전문적인 금융 데이터 소스가 필요합니다. 본 프로젝트에서는 야후 파이낸스(Yahoo Finance)의 데이터를 무료로 제공하는 오픈소스 라이브러리인 **yfinance** 를 사용합니다.

yfinance 는 주가(OHLCV), 기업 재무 정보, 애널리스트 추천 등 방대한 데이터를 제공하지만, 이 데이터는 주로 판다스(Pandas)의 데이터프레임(DataFrame)이나 복잡한 딕셔너리 형태로 반환됩니다. LLM 은 텍스트 기반 모델이므로, 이러한 이진(Binary) 혹은 객체 데이터를 그대로 전달하면 이해하지 못하거나 토큰을 과도하게 소모할 수 있습니다. 따라서 **데이터**

직렬화(Serialization) 과정이 필수적입니다.

데이터프레임을 마크다운(Markdown) 테이블 형식으로 변환하는 전략을 제시합니다. 마크다운 테이블은 텍스트로 데이터를 구조화하여 표현하는 가장 효율적인 방법 중 하나이며, GPT 모델은 마크다운 문법을 학습 과정에서 많이 접했기 때문에 이를 매우 잘 해석합니다.

3.2 핵심 금융 분석 함수 구현

sec02/gpt_functions.py

```
from datetime import datetime
import pytz
import yfinance as yf

def get_current_time(timezone: str = 'Asia/Seoul'):
    tz = pytz.timezone(timezone) # 타임존 설정
    now = datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")
    now_timezone = f'{now} {timezone}'
    print(now_timezone)
    return now_timezone

def get_yf_stock_info(ticker: str):
    stock = yf.Ticker(ticker)
    info = stock.info
    print(info)
    return str(info)

def get_yf_stock_history(ticker: str, period: str):
    stock = yf.Ticker(ticker)
    history = stock.history(period=period)
    history_md = history.to_markdown() # 데이터프레임을 마크다운 형식으로 변환
    print(history_md)
    return history_md

def get_yf_stock_recommendations(ticker: str):
    stock = yf.Ticker(ticker)
```

```

recommendations = stock.recommendations
recommendations_md = recommendations.to_markdown() # 데이터프레임을 마크다운
형식으로 변환
print(recommendations_md)
return recommendations_md

```

```

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_time",
            "description": "해당 타임존의 날짜와 시간을 반환합니다.",
            "parameters": {
                "type": "object",
                "properties": {
                    "timezone": {
                        "type": "string",
                        "description": "현재 날짜와 시간을 반환할 타임존을 입력하세요. (예:
Asia/Seoul)',
                    },
                },
                "required": ['timezone'],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "get_yf_stock_info",

```

```

    "description": "해당 종목의 Yahoo Finance 정보를 반환합니다.",
    "parameters": {
      "type": "object",
      "properties": {
        "ticker": {
          "type": "string",
          "description": "Yahoo Finance 정보를 반환할 종목의 티커를 입력하세요. (예: AAPL)',
        },
      },
      "required": ['ticker'],
    },
  },
  {
    "type": "function",
    "function": {
      "name": "get_yf_stock_history",
      "description": "해당 종목의 Yahoo Finance 주가 정보를 반환합니다.",
      "parameters": {
        "type": "object",
        "properties": {
          "ticker": {
            "type": "string",
            "description": "Yahoo Finance 주가 정보를 반환할 종목의 티커를
입력하세요. (예: AAPL)',
          },
          "period": {
            "type": "string",

```

```

        'description': '주가 정보를 조회할 기간을 입력하세요. (예: 1d, 5d, 1mo, 1y,
5y)',
        },
    },
    "required": ['ticker', 'period'],
    },
}
},
{
    "type": "function",
    "function": {
        "name": "get_yf_stock_recommendations",
        "description": "해당 종목의 Yahoo Finance 추천 정보를 반환합니다.",
        "parameters": {
            "type": "object",
            "properties": {
                'ticker': {
                    'type': 'string',
                    'description': 'Yahoo Finance 추천 정보를 반환할 종목의 티커를
입력하세요. (예: AAPL)',
                },
            },
            "required": ['ticker'],
        },
    }
},
]

```

```

if __name__ == '__main__':

```

```
# get_current_time('America/New_York')
# info = get_yf_stock_info('AAPL')

get_yf_stock_history('AAPL', '5d')
print('----')
get_yf_stock_recommendations('AAPL')
```

3.2.1 기업 기본 정보 조회 (get_yf_stock_info)

이 함수는 특정 종목의 티커(Ticker, 예: TSLA)를 받아 기업의 개요를 반환합니다.

```
def get_yf_stock_info(ticker: str):
    stock = yf.Ticker(ticker)
    info = stock.info
    return str(info) # 딕셔너리를 문자열로 변환
```

stock.info 는 기업의 주소, 시가총액, PER(주가수익비율), 사업 요약 등을 포함하는 거대한 딕셔너리입니다. 이를 문자열로 변환하여 GPT 에게 전달하면, GPT 는 "테슬라는 어떤 회사야?"라는 질문에 대해 "테슬라는 전기차와 에너지 저장 시스템을 설계 및 제조하는 회사이며..."와 같이 유창하게 설명할 수 있게 됩니다.¹

3.2.2 주가 히스토리 조회 (get_yf_stock_history)

데이터 번역의 기술: LLM이 이해하는 언어로 말하기

The Challenge

'yfinance'는 데이터를 판다스 데이터프레임(DataFrame)이나 복잡한 딕셔너리 객체로 반환합니다. LLM은 이러한 구조화된 데이터를 직접 이해할 수 없습니다.

The Solution

데이터 직렬화 (Data Serialization) . 객체를 텍스트 형식으로 변환하는 과정이 필수적입니다.



Pandas DataFrame Object



Date	Open	High	Low	Close
2023-10-27	150.00	155.00	149.00	154.50
2023-10-26	151.00	155.00	149.00	154.50
2023-10-25	150.00	151.00	146.00	153.50
2023-10-24	151.00	153.00	146.00	152.50
2023-10-21	151.00	150.00	149.00	152.50
2023-10-20	152.00	155.00	142.00	152.00
2023-10-17	152.00	153.00	149.00	154.50

LLM-Readable Markdown Text

```
def get_yf_stock_history(ticker: str, period: str):  
    stock = yf.Ticker(ticker)  
    history = stock.history(period=period)  
    return history.to_markdown() # 이 부분이 핵심!
```

.to_markdown()는 데이터프레임을 LLM이 가장 잘 해석하는 형식 중 하나인 **마크다운 테이블**로 변환합니다. 이를 통해 LLM은 복잡한 시계열 주가 데이터를 정확히 분석하고 요약할 수 있습니다.

사용자가 "최근 일주일간 애플 주가 흐름 어때?"라고 물을 때 사용되는 함수입니다.

```
def get_yf_stock_history(ticker: str, period: str):  
    stock = yf.Ticker(ticker)  
    history = stock.history(period=period)  
    return history.to_markdown() # 마크다운 변환
```

여기서 period 매개변수(예: '1d', '5d', '1mo')를 통해 데이터의 기간을 조절합니다. to_markdown() 메서드를 사용하기 위해서는 tabulate 라이브러리가 설치되어 있어야 하며, 만약 설치되어 있지 않다면 ModuleNotFoundError 또는 ImportError 가 발생하므로 주의해야 합니다 (pip install tabulate).¹

3.2.3 투자 의견 종합 (get_yf_stock_recommendations)

전문가의 의견을 묻는 질문("지금 테슬라 사도 될까?")에 대응하기 위한 함수입니다.

```
def get_yf_stock_recommendations(ticker: str):  
    stock = yf.Ticker(ticker)  
    return stock.recommendations.to_markdown()
```

이 함수는 '강력 매수(Strong Buy)', '매수(Buy)', '보유(Hold)' 등의 투자의견 분포를 시계열로 반환합니다. GPT 는 이 표를 해석하여 "전문가들의 의견은 다소 엇갈리고 있습니다. 매수 의견도 있지만 매도 의견도 상당수 존재하므로 신중한 접근이 필요합니다"와 같이 균형 잡힌 조언을 생성할 수 있습니다.¹

3.3 코드 리팩토링(Refactoring)과 확장성 확보

함수의 개수가 늘어남에 따라 메인 실행 파일(stock_info_streamlit.py)의 if-else 분기문이 복잡해질 수 있습니다. 연구 자료에서는 이를 간결하게 유지하기 위한 리팩토링 기법을 제안합니다.

각 도구의 실행 로직을 별도의 조건문 블록으로 분리하되, 결과 처리 부분(메시지 추가 등)을 공통화하여 코드의 중복을 줄입니다. 예를 들어, get_current_time, get_yf_stock_info 등 어떤 함수가 호출되든 그 결과(func_result)를 변수에 담고, 메시지를 추가하는 코드는 한 번만 작성하는 방식입니다.

리팩토링 예시

```
if tool_name == "get_current_time":  
    func_result = get_current_time(...)  
elif tool_name == "get_yf_stock_info":  
    func_result = get_yf_stock_info(...)  
#... (다른 함수들)
```

공통 메시지 처리

```
st.session_state.messages.append({  
    "role": "function",  
    "name": tool_name,  
    "content": func_result  
})
```

이러한 구조는 추후 재무제표 조회, 뉴스 검색 등 새로운 기능이 추가되더라도 코드를 쉽게 확장할 수 있게 해줍니다.¹

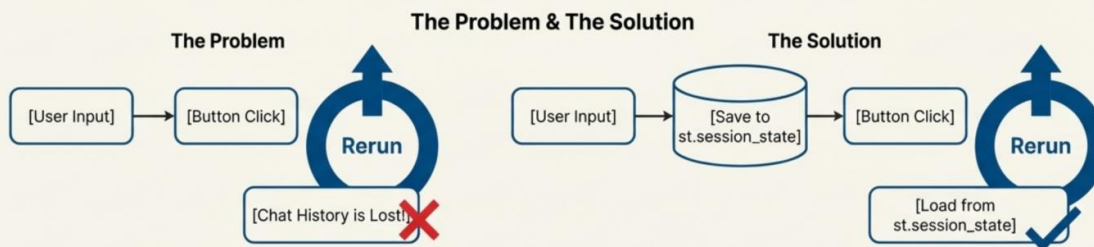
4. 고급 사용자 인터페이스: 스트림릿(Streamlit)과 반응형 웹

4.1 스트림릿의 실행 모델 이해

단순한 도구를 넘어 완성된 제품으로: Streamlit UI 구축

터미널 환경은 개발자에게 유용하지만, 일반 사용자를 위해 웹 기반의 UI가 필요합니다.
'Streamlit'은 파이썬만으로 빠르게 웹 앱을 만들 수 있는 강력한 도구입니다.

상호작용 시마다 스크립트 전체가 재실행(Rerun)됩니다. 이 때문에 일반 변수에 저장된 대화 내용은 버튼을 누를 때마다 사라집니다.



```
import streamlit as st

# 세션 상태가 없으면 메시지 리스트를 초기화합니다.
if "messages" not in st.session_state:
    st.session_state.messages = []
```

'st.session_state'는 재실행되어도 유지되는 특별한 저장소로, 챗봇의 대화 기록을 보존하는 데 필수적입니다.

터미널 환경은 개발자에게는 편리하지만 일반 사용자에게는 접근성이 떨어집니다. 따라서 웹 기반의 인터페이스를 구축하기 위해 파이썬 라이브러리인 '스트림릿(Streamlit)'을 사용합니다. 스트림릿은 HTML 이나 자바스크립트 없이 파이썬만으로 웹 앱을 만들 수 있게 해주지만, **상호작용 시마다 스크립트 전체가 재실행된다(Rerun)**는 독특한 실행 모델을 가지고 있습니다. 이 때문에 챗봇의 대화 내용을 유지하기 위해서는 st.session_state 라는 세션 저장소를 활용해야 합니다.

if "messages" not in st.session_state:

st.session_state.messages =

이 코드는 앱이 처음 실행될 때만 메시지 저장소를 초기화하고, 이후 버튼 클릭 등으로 앱이 재실행되더라도 대화 기록이 유지되도록 보장합니다.¹

4.2 사용자 경험(UX) 최적화

스트림릿 구현 시 고려해야 할 중요한 UX 요소 중 하나는 시스템 메시지와 함수 출력의 노출 여부입니다.

GPT 와 시스템 간의 내부적인 대화(함수 호출 요청, 함수 실행 결과 JSON 등)는 사용자에게 굳이 보여줄 필요가 없으며, 오히려 혼란을 줄 수 있습니다. 따라서 연구 자료에서는 `st.chat_message` 를 사용하여 대화 내용을 출력할 때, 메시지의 역할이 `user` 이거나 `assistant` 인 경우에만 화면에 표시하도록 필터링하는 로직을 제안합니다.

```
for msg in st.session_state.messages:
    if msg["role"] in ["user", "assistant"]:
        st.chat_message(msg["role"]).write(msg["content"])
```

이렇게 하면 사용자는 자신의 질문과 AI 의 최종 답변만을 깔끔하게 볼 수 있으며, 내부적으로 수행된 복잡한 데이터 조회 과정은 숨겨지게 됩니다.

5. 최적화 기술: 스트리밍(Streaming) 출력 구현

5.1 스트리밍의 필요성과 SSE(Server-Sent Events)

기다림의 경험을 바꾸다: 스트리밍 출력으로 UX 최적화

The Problem: 금융 데이터 분석처럼 복잡한 작업은 AI의 답변 생성까지 10초 이상 걸릴 수 있습니다. 이 시간 동안 화면이 멈춰 있으면 사용자는 앱이 멈췄다고 생각하고 이탈할 수 있습니다.

The Solution: 스트리밍 (Streaming). AI가 답변을 토큰 단위(token-by-token)로 생성되는 즉시 사용자에게 보여주는 기술입니다.



Implementation

- OpenAI API 호출 시 `stream=True` 매개변수를 추가합니다.
- API는 전체 응답 대신, 데이터 조각인 '청크(Chunk)'를 지속적으로 보내주는 제너레이터(Generator)를 반환합니다.

금융 데이터 분석과 같이 처리 시간이 긴 작업의 경우, 사용자가 답변을 받기까지 10 초 이상 소요될 수 있습니다. 정적인 UI에서는 이 시간 동안 화면이 멈춘 것처럼 보여 사용자가 이탈할 가능성이 높습니다. 이를 해결하기 위해 **스트리밍(Streaming)** 기술을 도입합니다. 스트리밍은 AI가 답변을 모두 완성할 때까지 기다리지 않고, 생성되는 즉시 한 글자씩(Token by token) 클라이언트로 전송하는 방식입니다.²

오픈 AI API에서는 `stream=True` 매개변수를 설정하여 이 기능을 활성화할 수 있습니다. 이때 API는 전체 응답 객체 대신, 조각난 데이터인 '청크(Chunk)'를 지속적으로 보내주는 제너레이터(Generator)를 반환합니다.

5.2 파이썬 제너레이터와 yield

스트리밍 응답을 처리하기 위해 함수는 일반적인 `return` 대신 `yield` 를 사용해야 합니다.

```
def get_ai_response(..., stream=True):  
    response = client.chat.completions.create(..., stream=stream)  
    for chunk in response:
```

yield chunk

이렇게 하면 get_ai_response 함수는 실행을 멈추지 않고 데이터를 실시간으로 호출자에게 전달하게 됩니다.

5.3 스트리밍 환경에서의 평선 콜링 난제

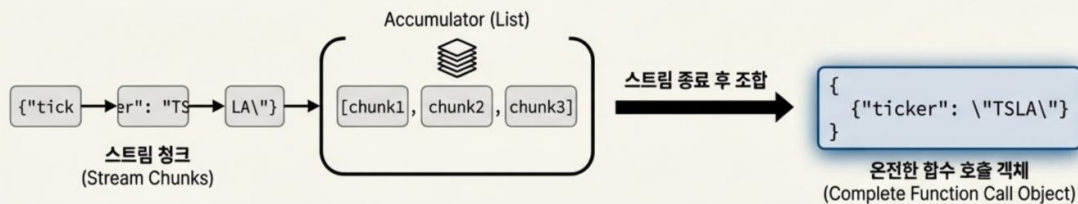
스트리밍의 가장 큰 난제: 조각난 평선 콜 데이터 재조립하기

The Core Problem: 스트리밍 모드에서는 함수 호출 정보(`tool\_calls`)마저 여러 조각으로 나뉘어 들어옵니다.

- 예시: {"ticker": "TSLA"}라는 JSON이 {"tick -> er": "TS -> LA"}와 같이 조각나서 수신됩니다.
- 조각난 상태에서는 JSON 파싱이 불가능하여 함수를 실행할 수 없습니다.

The Solution: 어큐레이터 (Accumulator) 패턴

- 조각난 청크들을 임시 리스트에 계속해서 누적(accumulate)합니다.
- 스트림이 끝나면, 누적된 조각들을 하나로 합쳐 **온전한 함수 호출 객체**로 복원합니다.



Conceptual Code Logic

```
tool_calls_chunk = []
# Stream loop starts
for chunk in response:
    if chunk.choices[0].delta.tool_calls:
        tool_calls_chunk.extend(chunk.choices[0].delta.tool_calls)

# After loop, assemble the complete tool call object
full_tool_call = assemble_from_chunks(tool_calls_chunk)
# Now, execute the function
```

가장 기술적으로 난이도가 높은 부분은 '스트리밍 모드에서의 평선 콜링 처리'입니다. 일반 텍스트 응답과 달리, 함수 호출 정보(함수명, 매개변수 JSON)도 스트리밍으로 조각나서 들어오기 때문입니다.

예를 들어, {"ticker": "TSLA"}라는 매개변수가 전송될 때, 첫 번째 청크에는 {"tick, 두 번째 청크에는 er": "TS, 세 번째 청크에는 LA"}와 같이 들어올 수 있습니다. 이 조각난 상태에서는 JSON 파싱이 불가능하므로 함수를 실행할 수 없습니다.

5.3.1 어큐레이터(Accumulator) 패턴

이 문제를 해결하기 위해 **어큐레이터(누적기) 패턴**을 사용해야 합니다. tool_calls_chunk 라는 리스트를 만들어 조각난 데이터를 모아두고, 이를 합쳐서 온전한 함수 호출 객체로 복원하는 과정을 상세히 설명합니다.¹

이를 위해 오픈 AI 커뮤니티에서 제안된 tool_list_to_tool_obj 라는 헬퍼 함수를 도입합니다. 이 함수는 분산된 델타(Delta) 정보를 인덱스별로 정렬하고 병합하여, 스트리밍이 아닌 일반 호출 때와

동일한 형태의 딕셔너리로 변환해줍니다.

```
# 스트리밍 데이터 조립 로직
```

```
if chunk.choices.delta.tool_calls:
```

```
    tool_calls_chunk += chunk.choices.delta.tool_calls
```

```
# 루프 종료 후 또는 실행 시점에서 조립
```

```
tool_obj = tool_list_to_tool_obj(tool_calls_chunk)
```

이 로직이 완벽하게 구현되어야만 스트리밍 환경에서도 안정적으로 주가 조회나 시간 확인 함수를 실행할 수 있습니다.

5.4 스트림릿에서의 실시간 렌더링

스트림릿 화면에서도 타자기 효과처럼 글자가 하나씩 나타나게 하려면 `st.empty()` 컨테이너를 활용해야 합니다.

1. 빈 컨테이너 생성: `box = st.empty()`
2. 청크 수신 루프 내에서 텍스트 누적: `full_text += chunk_content`
3. 컨테이너 업데이트: `box.markdown(full_text)`

이 과정을 반복하면 사용자는 AI가 실시간으로 답변을 작성하는 듯한 시각적 피드백을 받게 되어, 대기 시간의 지루함을 덜 수 있습니다. 연구 자료에서는 평선 콜링이 발생할 때 잠시 빈 칸이 생기는 문제를 해결하기 위해, 빈 컨테이너에 "함수 실행 중: `get_current_time`"과 같은 상태 메시지를 출력하는 UX 개선안도 포함하고 있습니다

sec02/stock_info_streamlit.py

```
from gpt_functions import get_current_time, tools, get_yf_stock_info, get_yf_stock_history,
get_yf_stock_recommendations
from openai import OpenAI
from dotenv import load_dotenv
import os
import json
import streamlit as st

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # 오픈 AI 클라이언트의 인스턴스 생성

def get_ai_response(messages, tools=None):
    response = client.chat.completions.create(
        model="gpt-4o", # 응답 생성에 사용할 모델 지정
        messages=messages, # 대화 기록을 입력으로 전달
        tools=tools, # 사용 가능한 도구 목록 전달
    )
    return response # 생성된 응답 내용 반환

st.title("💬 Chatbot")

if "messages" not in st.session_state:
    st.session_state["messages"] = [
        {"role": "system", "content": "너는 사용자를 도와주는 상담사야."}, # 초기 시스템 메시지
    ]
```

```

for msg in st.session_state.messages:
    if msg["role"] == "assistant" or msg["role"] == "user": # assistant 혹은 user 메시지인
        경우만
            st.chat_message(msg["role"]).write(msg["content"])

if user_input := st.chat_input(): # 사용자 입력 받기
    st.session_state.messages.append({"role": "user", "content": user_input}) # 사용자
    메시지를 대화 기록에 추가
    st.chat_message("user").write(user_input) # 사용자 메시지를 브라우저에서도 출력

    ai_response = get_ai_response(st.session_state.messages, tools=tools)
    ai_message = ai_response.choices[0].message
    print(ai_message) # gpt 에서 반환되는 값을 파악하기 위해 임시로 추가

    tool_calls = ai_message.tool_calls # AI 응답에 포함된 tool_calls 를 가져옵니다.
    if tool_calls: # tool_calls 가 있는 경우
        for tool_call in tool_calls:
            tool_name = tool_call.function.name # 실행해야한다고 판단한 함수명 받기
            tool_call_id = tool_call.id # tool_call 아이디 받기
            arguments = json.loads(tool_call.function.arguments) # 문자열을 딕셔너리로
            변환

            if tool_name == "get_current_time":
                func_result = get_current_time(timezone=arguments['timezone'])
            elif tool_name == "get_yf_stock_info":
                func_result = get_yf_stock_info(ticker=arguments['ticker'])
            elif tool_name == "get_yf_stock_history": # get_yf_stock_history 함수 호출
                func_result = get_yf_stock_history(

```

```

        ticker=arguments['ticker'],
        period=arguments['period']
    )
    elif tool_name == "get_yf_stock_recommendations": #
get_yf_stock_recommendations 함수 호출
        func_result = get_yf_stock_recommendations(
            ticker=arguments['ticker']
        )

    st.session_state.messages.append({
        "role": "function",
        "tool_call_id": tool_call_id,
        "name": tool_name,
        "content": func_result,
    })

    st.session_state.messages.append({"role": "system", "content": "이제 주어진 결과를
바탕으로 답변할 차례다."})

    ai_response = get_ai_response(st.session_state.messages, tools=tools) # 다시 GPT 응답
받기
    ai_message = ai_response.choices[0].message

    st.session_state.messages.append({
        "role": "assistant",
        "content": ai_message.content
    }) # AI 응답을 대화 기록에 추가합니다.

    print("AIWt: " + ai_message.content) # AI 응답 출력
    st.chat_message("assistant").write(ai_message.content) # 브라우저에 메시지 출력

```



Chatbot



테슬라 회사에 대해 알고 싶어



테슬라는 미국 텍사스주 오스틴에 본사를 두고 있으며, 전 세계적으로 전기차와 에너지 생성 및 저장 시스템을 설계, 개발, 제조, 임대 및 판매하는 회사입니다. 회사는 주로 두 가지 부문에서 운영됩니다: 전기차 사업과 에너지 생성 및 저장 사업입니다.

1. 전기차 사업:

- 전기차를 판매하고, 자동차 규제 크레딧을 판매하는 것 외에도 비보중 애프터 세일즈 차량, 중고차, 바디샵 및 부품, 슈퍼차징, 소매 상품, 차량 보험 서비스를 제공합니다.
- 회사는 직접 판매 및 네트워크를 통해 세단 및 SUV를 제공하며, Tesla Superchargers와 앱 업그레이드를 통해 구매 금융 및 리스 서비스를 제공합니다.

2. 에너지 생성 및 저장 사업:

- 태양 에너지 생성 및 에너지 저장 제품의 설계, 제조, 설치, 판매 및 리스를 하고 있습니다.
- 주거지, 상업지, 산업 고객 및 유틸리티를 대상으로 웹사이트, 매장, 갤러리 및 채널 파트너를 통해 서비스를 제공합니다.

테슬라의 주요 인물로는 공동 창립자이자 CEO인 엘론 머스크가 있으며, 회사는 지속 가능한 에너지로의 전환을 가속화하는 것을 목표로 하고 있습니다.

테슬라 주식은 나스닥 시장에서 TSLA라는 심볼로 거래되며 현재 주가는 약 451.45입니다. 회사의 최근 52주 최고가는 488.54, 최저가는 \$214.25입니다.

Your message



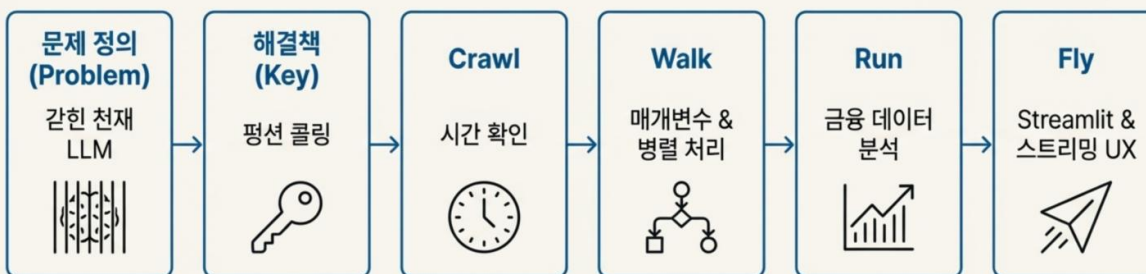
6. 결론: AI 에이전트 개발의 시사점

본 보고서를 통해 우리는 단순한 텍스트 생성기였던 LLM 을, 시간을 인지하고 금융 데이터를 분석하며 사용자와 실시간으로 소통하는 '에이전트'로 진화시키는 과정을 살펴보았습니다.

이 과정에서 **평션 콜링**은 LLM 과 현실 세계를 연결하는 가장 강력한 인터페이스임이 증명되었습니다. 또한 yfinance 와 같은 도메인 특화 라이브러리와 결합은 범용 AI 를 전문가 시스템으로 탈바꿈시키는 핵심 전략입니다. 마지막으로 스트리밍 기술과 스트림릿을 활용한 UI 구현은 복잡한 백엔드 로직을 사용자 친화적인 경험으로 포장하는 필수적인 엔지니어링 역량임을 확인했습니다.

앞으로 이러한 에이전트 구조는 포트폴리오 최적화, 자동 매매 실행, 뉴스 감성 분석 등 더욱 고도화된 금융 서비스로 확장될 수 있으며, 이는 핀테크 산업의 미래를 재정의하는 기술적 토대가 될 것입니다.

여정의 요약: 아이디어에서 완성된 AI 에이전트까지



Key Learnings Recap

- 평션 콜링은 LLM과 현실 세계를 연결하는 가장 강력한 인터페이스입니다.
- 데이터 직렬화 (.to_markdown())는 구조화된 데이터를 LLM이 이해할 수 있는 언어로 번역하는 핵심 기술입니다.
- 상태 관리 (st.session_state)는 Streamlit과 같이 재실행 모델을 가진 프레임워크에서 필수적입니다.
- 스트리밍과 어큐레이터 패턴은 복잡하지만 최상의 사용자 경험을 제공하기 위한 고급 최적화 기법입니다.

기술적 문제 해결 (Troubleshooting)

A.1 ModuleNotFoundError: No module named 'tabulate'

- **원인:** 판다스 데이터프레임의 `.to_markdown()` 메서드는 내부적으로 `tabulate` 라이브러리를 의존성으로 가집니다. 하지만 이는 판다스 설치 시 기본으로 포함되지 않는 경우가 많습니다.
- **해결:** 터미널에서 `pip install tabulate` 명령어를 실행하여 해당 라이브러리를 명시적으로 설치해야 합니다.¹

A.2 AttributeError: 'generator' object has no attribute 'choices'

- **원인:** `stream=True` 로 설정하면 반환되는 객체가 `ChatCompletion` 객체가 아니라 제너레이터(`Generator`)로 변경됩니다. 따라서 기존처럼 `response.choices` 로 직접 접근하면 속성 오류가 발생합니다.
- **해결:** `for chunk in response:` 구문을 사용하여 제너레이터를 순회(`Iteration`)하며 데이터를 처리해야 합니다.¹

A.3 타임존 관련 환각 현상

- **원인:** 파이썬의 `datetime.now()`는 기본적으로 시스템 시간이나 UTC 를 반환합니다. LLM 이 이를 특정 도시의 시간으로 오인하거나 잘못 계산할 수 있습니다.
- **해결:** `pytz` 라이브러리를 사용하여 명시적인 타임존 객체를 생성하고, 함수 호출 시 `timezone` 매개변수를 강제함으로써 계산의 책임을 LLM 이 아닌 코드(Python)로 이관해야 합니다.¹